

# 目录

|                          |   |
|--------------------------|---|
| 工业轴承设备剩余寿命预测系统的实现：编码规范文档 | 2 |
| 文档信息                     | 2 |
| 修订记录                     | 2 |
| 1. 适用范围与原则               | 2 |
| 2. 命名规范                  | 2 |
| 2.1 命名空间规范               | 2 |
| 2.2 类、函数、变量命名规范          | 3 |
| 3. 目录结构规范                | 3 |
| 4. Python 代码风格规范         | 3 |
| 5. 注释规范                  | 4 |
| 5.1 文件头注释规范              | 4 |
| 5.2 函数注释规范               | 4 |
| 6. 异常处理规范                | 4 |
| 7. 日志规范                  | 5 |
| 8. 配置文件规范                | 5 |
| 9. 测试代码规范                | 5 |
| 10. Git 提交规范             | 5 |
| 11. 文档编写规范               | 5 |
| 11.1 课程文档与导出规范           | 5 |
| 12. 结论                   | 6 |

# 工业轴承设备剩余寿命预测系统的实现：编码规范文档

## 文档信息

| 项目    | 内容                    |
|-------|-----------------------|
| 文档名称  | 编码规范文档                |
| 文档编号  | SE-PHM-MID-11         |
| 文档阶段  | 中期                    |
| 项目名称  | 工业轴承设备剩余寿命预测系统的实现     |
| 课程    | 中国科学技术大学软件学院《软件工程》    |
| 指导老师  | zjf                   |
| 小组成员  | zyj、cyy、zdh、zy        |
| 文档负责人 | cyy                   |
| 参与编写  | zyj、zdh、zy            |
| 版本    | V3.0                  |
| 修订日期  | 2026-06-14            |
| 归档形式  | Markdown 源文件、PDF、DOCX |
| 内容基线  | 源码、测试、文档、提交和导出规范      |

## 修订记录

| 版本   | 日期         | 编写人 | 说明                                            |
|------|------------|-----|-----------------------------------------------|
| V1.0 | 2025-11-10 | 项目组 | 完成初版课程文档                                      |
| V2.0 | 2026-03-12 | 项目组 | 统一项目名称、技术基线和阶段交付内容                            |
| V3.0 | 2026-06-14 | 项目组 | 参考课程交付样例统一封面与归档格式，补充数据理解、技术难点、测试验收和 DOCX 交付信息 |

## 1. 适用范围与原则

本文档适用于本项目所有 Python 源码、测试代码、脚本和文档编写工作。规范目标包括：

1. 提高代码可读性和一致性。
2. 降低多人协作中的接口误解和风格漂移。
3. 满足软件工程课程对命名空间、注释和提交规范的要求。

## 2. 命名规范

### 2.1 命名空间规范

采用名字空间的编程语言时，统一使用：

USTC.SSE.BearingPrediction

例如:

USTC.SSE.BearingPrediction.dataset

USTC.SSE.BearingPrediction.training

## 2.2 类、函数、变量命名规范

1. 类名使用 PascalCase, 例如 BaseTrainer、BearingEntity。
2. 函数名和变量名使用有意义的英文蛇形命名, 例如 load\_entity、remaining\_life。
3. 常量使用全大写下划线命名, 例如 DEFAULT\_WINDOW\_SIZE。
4. 禁止使用无意义名称, 例如 tmp、a、b 作为长期保留变量名。

## 3. 目录结构规范

项目核心目录应遵循统一职责划分。

src/USTC/SSE/BearingPrediction/

dataset/

preprocess/

feature/

labeling/

models/

training/

prediction/

evaluation/

visualization/

测试代码统一放在 tests/ 下, 课程提交文档放在 docx/ 下, 其他技术说明放在 docs/ 下。

## 4. Python 代码风格规范

1. 遵循 PEP 8。
2. 明确导入顺序: 标准库、第三方库、项目内模块。
3. 函数应尽量保持单一职责。
4. 优先使用显式返回值和清晰类型注解。

示例:

```
from pathlib import Path
```

```
import pandas as pd
```

```
from USTC.SSE.BearingPrediction.data.entities import BearingEntity
```

```
def load_metadata(metadata_path: Path) -> pd.DataFrame:
```

```
    """
```

```
    Load metadata from a csv file.
```

```
    Parameters
```

```
    -----
```

```
    metadata_path : Path
```

```
        metadata file path
```

```

Returns
-----
pd.DataFrame
    parsed metadata
"""

return pd.read_csv(metadata_path)

```

## 5. 注释规范

### 5.1 文件头注释规范

每个独立程序开头必须包含作者姓名、编程日期、程序主要功能和 USTC 版权说明。  
示例：

```

"""
Trainer module

this file is for model training with callbacks and experiment recording

created by zyj

copyright USTC

2026
"""

```

### 5.2 函数注释规范

主要函数或方法前必须写明功能、输入参数和输出参数。  
示例：

```

def extract_time_features(signal: np.ndarray) -> dict[str, float]:
    """
    Extract time domain features from a signal window.

    Parameters
    -----
    signal : np.ndarray
        input vibration signal

    Returns
    -----
    dict[str, float]
        extracted feature dictionary
    """

```

## 6. 异常处理规范

1. 对数据路径、文件格式和训练异常给出明确错误信息。
2. 禁止使用空的 `except` 块吞掉异常。
3. 对可恢复场景可进行兼容处理，但必须保留日志或告警。

示例:

```
if not data_root.exists():  
    raise FileNotFoundError(f"dataset root does not exist: {data_root}")
```

## 7. 日志规范

1. 关键训练过程应记录 epoch、loss、主要指标和梯度范数。
2. 梯度异常、训练提前停止和导出失败必须输出告警信息。
3. TensorBoard 日志目录应清晰可定位。

## 8. 配置文件规范

1. 训练配置应集中管理，不应将超参数散落在多个脚本中。
2. 实验配置建议导出为 JSON 或 YAML。
3. 配置中应至少包含模型名、学习率、权重衰减、batch size、epoch 数和采样策略。

## 9. 测试代码规范

1. 测试框架统一采用 pytest。
2. 测试函数命名采用 test\_ 前缀。
3. 优先使用合成数据减少环境依赖。
4. 测试代码也应保持清晰命名和必要注释。

## 10. Git 提交规范

1. 提交信息采用 type(scope): summary 格式。
2. 常用类型包括 feat、fix、chore、docs、test。
3. 单次提交应聚焦一个明确主题，避免把无关改动混在一起。

示例:

```
feat(cyy): add data pipeline and feature engineering  
docs(zy): reorganize proposal and mid-term deliverables
```

## 11. 文档编写规范

1. 文档统一使用 Markdown。
2. 项目名称统一为“工业轴承设备剩余寿命预测系统的实现”。
3. 技术栈统一写为 Python + uv 环境管理。
4. 术语、模块划分和时间安排必须与需求和设计文档一致。

### 11.1 课程文档与导出规范

1. 课程正式文档优先维护 Markdown 源文件，再通过脚本导出 PDF 和 DOCX，避免只修改二进制 Word 文件导致无法审阅差异。
2. 文档中项目名称统一为“工业轴承设备剩余寿命预测系统的实现”，成员名称统一为 zyj、cyy、zdh、zy。
3. 业务主线统一表述为“预测性维护、轴承健康评估、剩余寿命预测、生存概率/失效风险分析”，不把项目主线写成离散状态识别。
4. 引用论文复现实验时必须说明训练规模、数据划分和复现边界，避免把课程小规模训练说成作者完整实验结果。

5. 正式交付前执行导出脚本，并检查每份 Markdown 都存在对应 PDF 和 DOCX。

## 12. 结论

编码规范的目标不是增加形式负担，而是降低维护成本和沟通成本。对于本项目这种需要多人协作、同时交付代码与课程文档的场景，统一规范是保障项目质量和可提交性的必要条件。